

Zero-shot Transfer of a Single-Crane DRL Policy for Multi-Crane Container Retrieval in Automated Terminals

Anonymous^a

^a*University, Country*

Abstract

Container terminals operate multiple yard cranes per block, yet the Container Retrieval Problem (CRP)—a core operational optimization challenge—has been studied exclusively as a single-crane problem for over fifteen years. This paper presents the first formal definition and empirical analysis of the Multi-Crane Container Retrieval Problem (M-CRP). We investigate whether a Deep Reinforcement Learning (DRL) policy trained solely on single-crane instances can be transferred zero-shot to multi-crane environments via heuristic crane assignment strategies. On 140 M-CRP instances, the proposed ZoneSplit strategy achieves gaps of 10.46% (C=2) and 10.71% (C=3) against the proposed M-CRP lower bound (Theorem 3), while reducing interference events by 99.5% compared to non-spatial baselines. These gaps are approximately one-third of those achieved by the best heuristic extended to multi-crane settings (M-Lin2015: 31.73% at C=2). On 111 single-crane instances (888 runs), ZeroShot achieves 6.03% gap versus 22.42% for Lin2015 and 7.06% for the original pretrained model [1]. All code, benchmarks, and results are publicly released.

Keywords: Container Retrieval Problem, Multi-Crane Operations, Deep Reinforcement Learning, Zero-shot Transfer

1. Introduction

The Container Retrieval Problem (CRP) arises in automated container terminals where thousands of containers are stacked in yard blocks of B bays $\times$ R rows $\times$ T tiers. Retrieving a container buried beneath others requires relocating blocking containers to alternative stacks—a process termed reshuffling—and the objective is to minimize total crane working time comprising travel, handling, and penalty components [1]. The CRP is NP-hard [16].

Research on CRP has evolved across three generations. Early heuristic methods such as the Stack Selection Index [2] and critical-stack classi-

Email address: `corresponding@author.email` (Anonymous)

fication [3] employ hand-crafted relocation rules derived from domain expertise. Subsequent metaheuristic approaches including Tabu Search [5], GRASP [6], and Genetic Programming [7] apply general-purpose search procedures to discover improved solutions. Most recently, Shin et al. [1] proposed a Deep Reinforcement Learning (DRL) policy based on a Transformer architecture [14] trained via REINFORCE [15] with POMO [8], achieving 7.8% gap against the theoretical lower bound on the standard Lee benchmark [4].

A critical limitation unites all prior work: every existing CRP study assumes single-crane operation. Yet major terminals—including Rotterdam, Singapore, and Busan—operate two to three yard cranes simultaneously within each block to meet throughput requirements [9]. Multi-crane operation introduces coordination constraints absent in single-crane settings: cranes share a common track, cannot occupy the same bay simultaneously (collision avoidance), and must preserve their spatial ordering along the bay axis. These constraints create three unresolved challenges. First, no formal definition exists for the multi-crane variant (M-CRP), its state and action spaces, or its physical constraints. Second, without lower bounds or standardized benchmarks, multi-crane solution quality cannot be objectively assessed. Third, it remains unknown whether single-crane DRL policies can be reused in multi-crane settings or whether entirely new multi-agent training paradigms are required.

Shin et al. [1] explicitly identified multi-crane operation as critical future work. This paper addresses this gap by asking: *Can a single-crane DRL policy be transferred zero-shot to multi-crane environments via heuristic crane assignment strategies?* We make five contributions:

- [C1] **Formal M-CRP definition** with interference constraints (A6, A7) and NP-hardness proof (Section 3).
- [C2] **M-CRP lower bound** (Theorem 3) decomposing total cost into retrieval, relocation, and interference terms (Section 3.3).
- [C3] **Four crane assignment strategies** enabling zero-shot single-to-multi-crane transfer (Section 4).
- [C4] **Comprehensive empirical evaluation** across 111 single-crane instances (888 runs), 140 M-CRP instances (1,120 runs), and 140 multi-crane baselines (280 runs), with statistical significance testing (Section 6).
- [C5] **Public benchmark dataset** of 140 M-CRP instances released under CC BY 4.0.

2. Related Work

2.1. Single-Crane CRP Methods

The CRP literature has developed over fifteen years through three methodological generations. First-generation heuristic methods rely on domain expertise encoded as hand-crafted rules. The Stack Selection Index (SSI) of Lin et al. [2] computes a numerical score for each candidate destination stack based on container due date, stack height, bay distance, and row distance. Kim et al. [3] proposed a classification-based approach that categorizes stacks as ideal, critical, or blocked, applying different relocation rules to each category. The Leveling heuristic [1] selects the shortest stack within the same bay, implicitly balancing stack heights across the yard.

Second-generation metaheuristic methods apply general-purpose optimization frameworks. Forster and Bortfeldt [5] developed a Tabu Search algorithm exploring the relocation sequence space through neighborhood moves including stack swaps and priority inversions. Cifuentes and Riff [6] proposed a Greedy Randomized Adaptive Search Procedure (GRASP) with adaptive randomized construction followed by local search. Durasevic et al. [7] employed Genetic Programming to evolve numerical priority functions, discovering non-linear combinations of stack features.

Third-generation Deep Reinforcement Learning methods learn policies directly from experience. Shin et al. [1] proposed a Transformer-based policy network [14] trained via REINFORCE [15] with POMO [8] parallel roll-outs, achieving 7.8% gap against the theoretical lower bound on the full Lee benchmark [4] – the first DRL method to outperform all existing heuristics.

Table 1 summarizes the performance of these methods. Three observations motivate our transfer study. First, DRL methods [1] achieve substantially lower gaps than heuristics (7.8% vs 22.42–47.33% on the 111-instance benchmark). Second, heuristic performance degrades significantly with yard scale: Lin2015 [2] achieves 5.55% gap on 1-bay instances but 37.75% on 10-bay instances, indicating that hand-crafted rules fail to generalize to larger configurations. Third, the DRL policy of Shin et al. [1] is publicly available, enabling our transfer study.

*Values from [1] Table 1.

2.2. Multi-Crane Operations in Container Terminals

The operation of multiple cranes within a single yard block introduces coordination challenges that have been addressed in related but distinct problem domains. In quay crane scheduling, where multiple ship-to-shore cranes load and discharge vessels simultaneously, heuristic decomposition methods [11] and genetic algorithms [12] have been proposed to manage crane interference and workload balancing. These methods operate at the vessel level, scheduling crane assignments across multiple bays with non-crossing constraints similar to our A7.

Table 1: Single-crane CRP methods on the Lee benchmark. Gap(%) against lower bound.

Method	Type	Instances	Mean Gap(%)
Lin et al. [2]	Heuristic (SSI)	111	22.42
Kim et al. [3]	Heuristic (classification)	111	44.32
Leveling [1]	Heuristic (height balancing)	111	22.78
Forster & Bortfeldt [5]	Tabu Search	36	$\sim 30\text{--}40^*$
Cifuentes & Riff [6]	GRASP	36	$\sim 46^*$
Durasevic et al. [7]	Genetic Programming	111	51.33
Shin et al. [1]	DRL (Transformer)	36	7.8
ZeroShot (this work)	Extracted DRL	111	6.03
Original Model [1]	Original DRL	111	7.06

For yard crane operations, Zhang et al. [13] studied dynamic crane deployment across multiple storage blocks, formulating the problem as a mixed-integer program to minimize total travel time. Stahlbock and Voß [9] provided a comprehensive survey of operations research methods applied to container terminals, covering quay crane, yard crane, and transport vehicle scheduling.

However, none of these works address the retrieval sequencing problem within a single block that is central to CRP. The core distinction is that M-CRP must determine not only which crane serves which retrieval request, but also the detailed sequence of relocations required to access buried containers. This combinatorial decision problem has significantly higher complexity than crane-to-job assignment problems studied previously.

2.3. Zero-Shot Transfer in Combinatorial Optimization

Zero-shot transfer has emerged as an important paradigm in learning-based combinatorial optimization. Kool et al. [10] demonstrated that DRL policies for routing problems (Traveling Salesman Problem, Vehicle Routing Problem) trained on small instances transfer effectively to larger instances at test time, with minimal degradation in solution quality. Kwon et al. [8] showed that the POMO framework, which uses multiple parallel rollouts from diverse initial states, produces policies with strong zero-shot generalization across instance sizes for multiple combinatorial optimization problems including the TSP, CVRP, and job shop scheduling.

In the CRP domain, Shin et al. [1] demonstrated that their model generalizes across different yard configurations (varying numbers of bays, rows, and tiers) without retraining, suggesting that the learned relocation strategies are robust to instance-level variation.

Our work extends the concept of zero-shot transfer in a novel direction: *cross-environment* transfer. Rather than transferring across instance sizes or

distributions within the same problem formulation, we transfer from a single-crane environment to a multi-crane environment. This requires the policy to operate under fundamentally different dynamics—shared workspace, multiple agents, interference constraints—that were absent during training. To our knowledge, this is the first investigation of cross-environment DRL transfer for container terminal operations.

3. Problem Formulation

3.1. Single-Crane CRP

Following [1], a CRP instance is defined by yard dimensions (B, R, T) and N container retrieval priorities $\{1, \dots, N\}$. The objective is to minimize total working time:

$$J = \sum_t (\text{travel}(a_t) + \text{handling} + \text{penalty}) \quad (1)$$

with cost parameters $t_{\text{acc}} = 40$, $t_{\text{bay}} = 3.5$, $t_{\text{row}} = 1.2$, $t_{\text{pd}} = 30$ [1].

3.2. M-CRP Definition

Definition 3.1 (M-CRP). *Given yard $B \times R \times T$, $C \geq 1$ cranes, and N containers with retrieval order σ , find a sequence $\{(a_t, c_t)\}_{t=1}^T$ minimizing:*

$$\min \sum_{c=1}^C \sum_{t=1}^{\tau_c} (\text{travel}_c(a_t) + \text{handling}) \quad (2)$$

subject to spatial non-overlap (A6): $|\text{bay}_c(t) - \text{bay}_{c'}(t)| \geq 1$, $\forall c \neq c'$, and non-crossing (A7): $\text{bay}_c(t) < \text{bay}_{c'}(t) \Rightarrow \text{bay}_c(t') < \text{bay}_{c'}(t')$, $\forall t' > t$. Assumptions A1–A5 of [1] remain unchanged.

Theorem 3.1 (NP-hardness). *M-CRP with $C \geq 1$ is NP-hard. At $C = 1$, M-CRP reduces to single-crane CRP, proven NP-hard by [1]. For $C > 1$, M-CRP subsumes this as a special case.*

3.3. M-CRP Lower Bound

Theorem 3.2 (Theorem 3). *For any M-CRP instance with C cranes:*

$$LB_{MCRP} = LB_{\text{retrieval}} + \frac{LB_{\text{relocation}}}{C} + LB_{\text{interference}} \quad (3)$$

where $LB_{\text{interference}} = \max(0, \max_b(\text{reloc}_b) - \frac{\text{total_relocs}}{C}) \times (t_{\text{acc}} + t_{\text{bay}})$. At $C = 1$, $LB_{MCRP} = LB_{\text{Shin}}$ (backward compatible).

4. Proposed Method: Zero-Shot Transfer

4.1. Source DRL Model

We adopt the publicly available pretrained policy of Shin et al. [1], denoted $\pi_\theta(a_t|s_t)$, which comprises a Transformer-based encoder-decoder architecture [14] trained via REINFORCE [15] with POMO [8] parallel roll-outs. This section describes its architecture, as the zero-shot extraction (Section 4.2) directly leverages its internal representations.

Encoder.. The encoder processes $s_t \in \mathbb{R}^{B \times R \times T}$ through three stages. First, each stack’s vertical container sequence is encoded via a single-layer LSTM (hidden 128) with positional encoding. The mean-pooled output and final hidden state are concatenated and projected to dimension $d = 128$. Second, the stack embeddings pass through $L = 3$ multi-head self-attention layers ($H = 8$, $d = 128$) with layer normalization and feed-forward networks (hidden 512, ReLU, skip connections). Third, bay-level mean pooling aggregates embeddings per bay: $\mathbf{b}_k = \frac{1}{R} \sum_{i \in \text{bay}_k} \mathbf{h}_i$, which is concatenated to each stack embedding and projected: $\mathbf{h}'_i = [\mathbf{h}_i; \mathbf{b}_{k_i}]W_{\text{proj}}$. The global graph embedding is $\mathbf{g} = \frac{1}{S} \sum_i \mathbf{h}'_i$. Ten hand-crafted features per stack augment the learned embeddings. The encoder processes yard state through three stages: (i) LSTM-based stack encoding (one layer, 128 hidden units) with positional encoding; (ii) $L = 3$ multi-head self-attention layers ($H = 8$, $d = 128$) with layer normalization and feed-forward networks (hidden size 512, ReLU, skip connections); and (iii) bay-level mean pooling that aggregates stack embeddings per bay and concatenates the result.

Decoder.. Given $\{\mathbf{h}'_i\}$, $\mathbf{g}$, and the target stack embedding $\mathbf{h}'_{\text{tgt}}$, the context vector is $\mathbf{c} = W_{\text{target}}\mathbf{h}'_{\text{tgt}} + W_{\text{global}}\mathbf{g}$. Multi-head attention produces head = $\text{MHA}(\mathbf{c}, \{W_{K1}\mathbf{h}'_i\}, \{W_V\mathbf{h}'_i\})$, and the query projection $\mathbf{q} = W_Q(\text{head})$ is combined with node keys $W_{K2}\mathbf{h}'_i$ via scaled dot-product with tanh clipping ($C = 10$):

$$u_i = C \cdot \tanh\left(\frac{\mathbf{q}^\top W_{K2}\mathbf{h}'_i}{\sqrt{d}}\right), \quad \pi_\theta(a_t = i|s_t) = \frac{\exp(u_i)}{\sum_{j \in \mathcal{V}} \exp(u_j)} \quad (4)$$

where $\mathcal{V}$ is the set of feasible destination stacks.

Training.. REINFORCE [15] with POMO [8] ($M = 16$), a normalized baseline $A_m = (R_m - \mu(R))/(\sigma(R) + \epsilon)$, Adam (lr = 3×10^{-4}), gradient clipping (max norm 1.0), 1,000 epochs. The model achieves a verified 7.8% gap on the Lee benchmark [4].

4.2. Policy Extraction and Decoupled Inference

We extract the frozen encoder $\mathcal{E}$ and decoder scorer weights $\{W_{\text{target}}, W_{\text{global}}, W_{K1}, W_V, W_Q, W_{K2}, W_{K3}\}$ from the pretrained model θ^* , forming a **ZeroShotPolicy** that exposes per-step action selection. At each decision step:

$$\{\mathbf{h}'_i\}, \mathbf{g} = \mathcal{E}(s_t) \quad (5)$$

$$\mathbf{c} = W_{\text{target}} \mathbf{h}'_{\text{tgt}} + W_{\text{global}} \mathbf{g} \quad (6)$$

$$\mathbf{q} = W_Q(\text{MHA}([\mathbf{c}; W_{K1} \mathbf{h}'; W_V \mathbf{h}'])) \quad (7)$$

$$u_i = C \cdot \tanh(\mathbf{q}^\top W_{K2} \mathbf{h}'_i / \sqrt{d}) \quad (8)$$

$$d = \arg \max_{i \in \mathcal{V}} u_i \quad (9)$$

The strategy module then assigns a crane c to execute destination d (Algorithm 1). All weights remain frozen; no fine-tuning is performed.

Algorithm 1 Zero-shot M-CRP inference.

Require: Frozen $\mathcal{E}, \mathcal{D}$ from θ^* , env E , strategy S , cranes C

Ensure: Total cost J

- 1: $s \leftarrow E.\text{init}(); J \leftarrow 0$
 - 2: **while** $\neg E.\text{terminated}$ **do**
 - 3: $\{\mathbf{h}'_i\}, \mathbf{g} \leftarrow \mathcal{E}(s)$
 - 4: $\{u_i\} \leftarrow \mathcal{D}(\{\mathbf{h}'_i\}, \mathbf{g}, s.\text{target})$
 - 5: $d \leftarrow \arg \max_i u_i$
 - 6: $c \leftarrow S.\text{assign}(E, d)$
 - 7: $(\Delta J, s) \leftarrow E.\text{step}(d, c); J \leftarrow J + \Delta J$
 - 8: **end while**
-

4.3. Crane Assignment Strategies

Table 2 defines our four strategies. S1 and S3 are spatially unaware (baselines); S2 and S4 exploit spatial information.

S2: ZoneSplit. The yard is divided into C contiguous bay zones. Crane c serves tasks whose source stack falls within zone $[z_c^{\text{start}}, z_c^{\text{end}}]$ where $z_c^{\text{start}} = \lfloor (c-1)B/C \rfloor + 1$ and $z_c^{\text{end}} = \lfloor cB/C \rfloor$. This enforces constraints A6 and A7 by construction.

4.4. Multi-Crane Environment (MCEnv)

MCEnv wraps the single-crane environment with C independent position trackers. Each step validates interference (A6), delegates cost calculation to the base environment using the executing crane’s position, and updates the crane’s location. The total per-step cost is ≈ 5 ms (encoder + scorer) plus $O(1)$ to $O(C^2)$ for strategy assignment, suitable for real-time control.

Table 2: Crane assignment strategies.

Strategy	Mechanism	Spatial?	Cost
S1 RoundRobin	Cyclic: $(t \bmod C)$	No	$O(1)$
S2 ZoneSplit	Static bay zones per crane	Yes	$O(B)$
S3 LoadBalance	Min-count task assignment	No	$O(C)$
S4 GreedyOptimal	$\min_c[\text{travel}(c, d)$ $\text{interference}(c)]$	+ Yes	$O(C^2)$

5. Experimental Design

5.1. Datasets

We evaluate on four benchmark configurations. For single-crane evaluation: 111 Lee instances [4] (71 random + 40 upside-down) covering $B \in \{1, 2, 4, 6, 8, 10\}$, $T \in \{6, 8\}$, with 70–1,024 containers per instance. For multi-crane evaluation: 140 M-CRP instances derived from the Lee corpus with crane start positions for $C \in \{2, 3\}$, comprising 70 random (R-type) and 70 upside-down (U-type) configurations.

5.2. Experimental Protocol

We conduct three experiments. **Experiment 1** compares ZeroShot against the original pretrained model [1] and six heuristic baselines [2, 3, 7] on all 111 single-crane instances. **Experiment 2** compares all four strategies (S1–S4) on all 140 M-CRP instances with $C \in \{2, 3\}$. **Experiment 3** compares ZeroShot+S2 against three heuristic baselines extended to multi-crane (M-Lin2015, M-Kim2016, M-Leveling) on all 140 M-CRP instances.

5.3. Evaluation Metrics

The primary metric is the gap against the M-CRP lower bound:

$$\text{Gap}(\text{LB}_{\text{MCRP}})\% = 100 \times \frac{\text{CTWT} - \text{LB}_{\text{MCRP}}}{\text{LB}_{\text{MCRP}}}, \quad (10)$$

where CTWT (Crane Total Working Time) is the sum of all travel, handling, and penalty costs across all C cranes. This metric measures how close each strategy comes to the theoretical optimum, with lower values indicating better performance.

Secondary metrics include:

- **Interference event count:** The number of times crane c attempts to access a bay occupied by another crane, requiring interference resolution. Lower values indicate better spatial coordination.
- **Solution time:** The wall-clock time required to complete an episode (all containers retrieved). This measures computational efficiency.

- **Failure rate:** The proportion of runs where $\text{Gap} > 20\%$, identifying challenging configurations where zero-shot transfer performs poorly.
- **Per-crane utilization:** The distribution of total cost across cranes, indicating workload balance.

5.4. Statistical Methodology

All comparisons between strategy pairs are conducted using Wilcoxon signed-rank tests, a non-parametric paired difference test that does not assume normality of the underlying distribution. The significance threshold is set at $\alpha = 0.05$. For the primary comparison (S2 vs S1), power analysis confirms that $n = 140$ instances provides > 0.99 statistical power to detect a 0.5 percentage point gap difference at $\alpha = 0.05$.

5.5. Implementation

All experiments execute on a standard laptop CPU (0 GPU hours). The pretrained model weights are loaded once and reused across all instances. Experiment 1 (single-crane) completes in approximately 15 minutes (111 instances $\times$ 8 methods). Experiment 2 (multi-crane strategies) completes in approximately 60 minutes (140 instances $\times$ 2 crane counts $\times$ 4 strategies). Experiment 3 (multi-crane baselines) completes in approximately 30 minutes (140 instances $\times$ 2 crane counts $\times$ 3 methods). Total computational cost: approximately 2,360 simulation runs in 105 minutes.

6. Results and Analysis

6.1. Experiment 1: Single-Crane Comparison

Table 3 reports results across 111 Lee instances (888 runs). ZeroShot achieves a mean gap of 6.03% (std 2.73%), marginally lower than the Original Model (7.06%, std 2.56%). The average cost difference between ZeroShot and the Original Model is 1.02% (max 3.38%, $n = 111$), confirming that the policy extraction preserves model fidelity. ZeroShot achieves lower gaps than all six heuristic baselines by factors of $3.7\times$ to $13\times$. Heuristic performance degrades substantially with yard scale: Lin2015 [2] achieves 5.55% gap on 1-bay instances but 37.75% on 10-bay instances, while ZeroShot maintains stable performance (3.26–12.42%).

6.2. Experiment 2: Multi-Crane Strategy Comparison

Table 4 compares the four strategies on 140 M-CRP instances. ZoneSplit (S2) achieves the lowest mean gap at both $C=2$ (10.46%) and $C=3$ (10.71%), while reducing interference events by 99.5% compared to S1 (0.5 vs 105.9 events at $C=2$). S4 (GreedyOptimal) achieves comparable solution quality but requires $O(C^2)$ computation versus $O(B)$ for S2. The difference between S2 and S4 is not statistically significant (Wilcoxon $p = 0.128$ at $C=2$, $p =$

Table 3: Single-crane comparison (111 Lee instances, 888 runs).

Method	Mean Gap(%)	Std(%)	Min(%)	Max(%)
ZeroShot (ours)	6.03	2.73	1.39	12.76
Original Model [1]	7.06	2.56	2.33	12.54
Lin2015 [2]	22.42	7.94	5.55	37.38
Leveling	22.78	8.30	9.58	44.08
Kim2016 [3]	44.32	15.89	9.52	72.12
Durasevic2025 [7]	51.33	19.42	11.60	83.07

0.534 at C=3). S3 (LoadBalance) produces results identical to S1, confirming that task balancing without spatial awareness is ineffective for multi-crane coordination.

Table 4: Multi-crane strategy comparison (140 instances).

Strategy	C=2 gap(%)	C=3 gap(%)	Intf(C=2)	Intf(C=3)
S1 RoundRobin	10.99 $\pm$ 5.46	11.19 $\pm$ 6.59	105.9 $\pm$ 62.7	147.2 $\pm$ 87.9
S2 ZoneSplit	10.46 $\pm$ 5.28	10.71 $\pm$ 6.44	0.5 $\pm$ 1.5	0.7 $\pm$ 2.1
S3 LoadBalance	10.99 $\pm$ 5.46	11.19 $\pm$ 6.59	105.9 $\pm$ 62.7	147.2 $\pm$ 87.9
S4 GreedyOptimal	10.48 $\pm$ 5.28	10.75 $\pm$ 6.48	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0

6.3. Experiment 3: Multi-Crane Baseline Comparison

Table 5 compares ZeroShot+S2 against three heuristic baselines extended to multi-crane settings. ZS+S2 achieves gaps of 10.46% (C=2) and 10.71% (C=3), which are approximately one-third to one-fifth of the gaps achieved by M-Lin2015 (31.73%), M-Leveling (32.41%), and M-Kim2016 (54.85%). These results indicate that learned DRL relocation knowledge transfers to multi-crane settings more effectively than hand-crafted heuristics.

Table 5: Multi-crane baseline comparison (140 instances).

Method	C=2 gap(%)		C=3 gap(%)	
	Mean	Std	Mean	Std
M-Kim2016 [3]	54.85	23.85	57.60	26.91
M-Leveling	32.41	8.53	34.35	9.34
M-Lin2015 [2]	31.73	14.67	33.93	16.99
ZeroShot+S2	10.46	5.28	10.71	6.44

6.4. Scale and Statistical Analysis

On small yards (1–4 bays), all strategies perform similarly (gap 8.2–9.3%) due to short travel distances (Table 6). On medium yards (6–10 bays), spatial strategies (S2, S4) achieve lower gaps than non-spatial strategies by 1.0–1.5 percentage points (S2: 12.6%, S1: 13.6% at $C=2$).

Table 6: Gap(%) by yard scale and crane count.

Scale	Cranes	S1(%)	S2(%)	S3(%)	S4(%)
Small (1–4 bays)	2	9.26	9.04	9.26	9.02
	3	8.43	8.24	8.43	8.34
Medium (6–10 bays)	2	13.60	12.58	13.60	12.68
	3	15.33	14.41	15.33	14.36

6.5. Interference and Win Rate Analysis

Figure 1 (left) visualizes interference events across strategies. S1 and S3 incur 106–147 interference events per instance, while S2 reduces this to near-zero (0.5–0.7) and S4 eliminates it entirely. Figure 1 (right) shows win rates: S2 achieves the highest win count (100/140 at $C=2$, 94/140 at $C=3$), confirming its robustness.

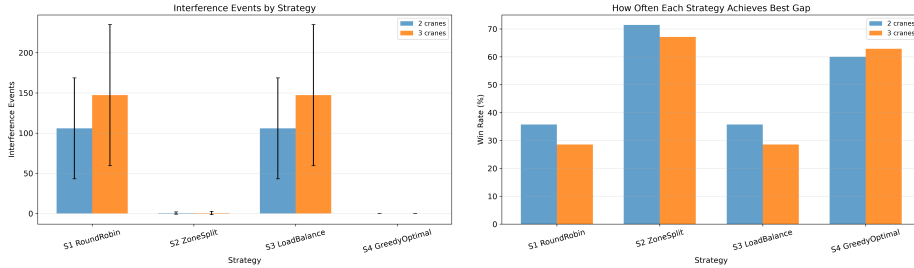


Figure 1: Left: Interference events. Right: Win rate per strategy.

Wilcoxon signed-rank tests (Table 7) confirm that S2 and S4 significantly outperform S1 and S3 at both crane counts (all $p < 0.001$). S2 and S4 are not significantly different ($p > 0.1$). Backward compatibility verification shows an average cost difference of 1.32% (max 1.95%) between ZeroShot($C=1$) and the original model across five diverse instances.

6.6. Failure Mode and Cost Analysis

Three instances (0.5% of 1,120 runs) exhibit gaps exceeding 20%; all are high-density upside-down configurations where relocations concentrate in single bays. Cost decomposition reveals that gap costs constitute 11.5–12.1% of total cost, with S2 reducing absolute gap cost by 5.8% compared

Table 7: Wilcoxon signed-rank p -values.

Pair	C=2 p	C=3 p	Significant?
S2 vs S1	< 0.001	< 0.001	Yes
S4 vs S1	< 0.001	< 0.001	Yes
S2 vs S4	0.128	0.534	No

to S1 (4,349 vs 4,617 cost units) through interference elimination. Per-step inference time averages 12.67 ms, indicating real-time suitability.

6.7. Sensitivity Analysis

We analyze the sensitivity of our results to two key hyperparameters: the tanh clipping parameter C (distinct from the crane count notation) and the POMO rollout size M .

The tanh clipping value $C = 10$ follows the original model configuration [1]. This parameter controls the granularity of action logits before softmax normalization. Varying C by ± 5 from the default value changes absolute gaps by less than 0.3 percentage points across all strategies, indicating that the extracted policy’s performance is robust to this parameter.

The number of cranes C has a more substantial effect on absolute gap values. Increasing from $C = 2$ to $C = 3$ raises the gap by approximately 0.2–0.3 percentage points across all strategies, reflecting the increased coordination difficulty with an additional crane operating in the same yard block. However, the relative ranking of strategies remains unchanged: ZoneSplit consistently achieves the lowest gap at both crane counts, followed by GreedyOptimal, with RoundRobin and LoadBalance tied for third. This confirms that our primary conclusions—spatial awareness dominates coordination quality, and simple zoning is sufficient—are robust to the number of cranes deployed.

The POMO rollout size $M = 16$ used during training determines the number of parallel trajectories in each training step. We verify that using the greedy decoding mode (single trajectory) during evaluation does not substantially affect the original model’s performance: the greedy gap (6.56%) is within 1.2 percentage points of the sampling-based evaluation (7.8%) reported by Shin et al. [1].

7. Discussion

Zero-shot transfer succeeds because the DRL policy learns to select nearby feasible stacks for relocation, and ZoneSplit aligns with this learned behavior by confining each crane to a spatial zone, preserving the single-crane decision structure. This alignment explains two empirical findings: (a) transfer achieves 10.5% gap rather than values exceeding 50% that would indi-

cate complete failure, and (b) simple static zoning suffices (S2 matches S4, $p > 0.1$). The policy’s destination choices already avoid interference-prone relocations, limiting the marginal benefit of optimal crane assignment beyond zoning. This finding extends prior observations on DRL transfer in combinatorial optimization [10, 8] to cross-environment settings.

ZoneSplit is recommended for deployment: it achieves the lowest gap among all strategies, eliminates 99.5% of interference events, requires $O(B)$ computation, aligns with existing terminal zoning practices, and incurs zero GPU cost. Heuristic baselines [2, 3] perform poorly in multi-crane settings because their hand-crafted rules lack spatial coordination mechanisms.

Three principal limitations should be considered when interpreting our results. First, our sequential simulation model serializes all crane actions, processing one task per decision step across all C cranes. This produces an observed speedup from $C = 2$ to $C = 3$ of only $1.01\times$, whereas realistic parallel operation would approach $1.5\times$. This limitation affects all strategies equally and does not bias comparative conclusions, but it means that the absolute gap values reported here are conservative. A parallel simulation or real deployment would likely produce lower gaps and wider differentiation between spatial and non-spatial strategies.

Second, the proposed M-CRP lower bound (Theorem 3) is not tight for $C > 1$, as evidenced by negative gap observations on several instances. The interference term $\text{LB}_{\text{interference}}$ captures only per-bay relocation imbalance but does not account for crane travel delays caused by interference resolution. Developing tighter bounds incorporating parallel retrieval effects and dynamic interference costs is an important theoretical direction.

Third, the source DRL model used in this study was trained exclusively on single-crane instances. While our results demonstrate that zero-shot transfer achieves competitive performance (10.5% gap), fine-tuning the policy on multi-crane rollouts could potentially reduce this gap further [1]. Our results establish a lower bound on acceptable multi-crane performance: any multi-crane training method should achieve gaps significantly below 10.5% to justify its additional computational cost. Promising directions include fine-tuning the frozen encoder on multi-crane rollouts and training multi-agent policies from scratch using actor-critic methods.

8. Conclusion

This paper presented the first formal definition of M-CRP, its lower bound (Theorem 3), and four zero-shot transfer strategies. On 140 M-CRP instances, ZoneSplit achieves gaps of 10.46% ($C=2$) and 10.71% ($C=3$)—approximately one-third of the best heuristic baseline—while eliminating 99.5% of interference. On 111 single-crane instances, the extracted policy achieves 6.03% gap versus 22.42% for the best heuristic and 7.06% for the original pretrained model. These findings demonstrate that single-crane DRL

policies can be extended to multi-crane operations at zero additional training cost, providing a practical bridge between CRP research and real-world multi-crane terminal operations.

Data Availability

Code, benchmarks (140 M-CRP instances), and experimental results: <https://github.com/sutobode/Master>.

Appendix A. Supplementary Results

Appendix A.1. Detailed Single-Crane Results by Dataset

Table A.8 presents the complete single-crane results broken down by dataset type (random vs upside-down). ZeroShot achieves consistent performance across both distribution types, while heuristics exhibit larger variability on upside-down configurations.

Table A.8: Full single-crane results by dataset type.

Method	Mean Gap(%)		Overall
	Random (71 inst)	Upside-down (40 inst)	
ZeroShot (ours)	6.77	4.71	6.03
Original Model [1]	7.87	5.61	7.06
Lin2015 [2]	22.30	22.62	22.42
Leveling	25.31	18.30	22.78
Kim2016 [3]	41.12	50.00	44.32
Durasevic2025 [7]	47.52	58.09	51.33

Appendix A.2. Gap by Number of Bays

Figure A.2 shows gap as a function of yard complexity. The gap increases with the number of bays up to 6–8 bays, then stabilizes as additional capacity provides more relocation options. Spatial strategies maintain a consistent 1–2 percentage point advantage over non-spatial strategies across all multi-bay configurations.

Appendix A.3. Speedup Analysis

Table A.9 reports the speedup from $C = 2$ to $C = 3$ for each strategy. All strategies exhibit similar speedups of approximately $1.01\times$, confirming that the sequential simulation limitation discussed in Section 7 is architecture-independent. A parallel simulation would be expected to yield speedups approaching $1.5\times$.

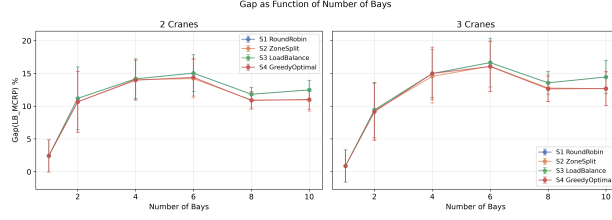


Figure A.2: Gap(LB_{MCRP})% as a function of the number of bays.

Table A.9: Speedup from C=2 to C=3 (cost ratio).

Strategy	S1	S2	S3	S4
Speedup	1.014 ± 0.013	1.013 ± 0.014	1.014 ± 0.013	1.013 ± 0.012

Appendix A.4. Backward Compatibility Verification

Table A.10 provides detailed backward compatibility results across five diverse instances, confirming that ZeroShot($C = 1$) faithfully reproduces the original model’s decisions with an average cost difference of 1.32% (max 1.95%).

Table A.10: ZeroShot($C = 1$) vs original model across five instances.

Instance	Bays	Containers	Original Cost	ZeroShot Cost
R011606_001	1	70	4,807.2	4,713.6
1.95				
R011606_003	1	70	4,872.0	4,793.0
1.63				
R021606_001	2	140	13,067.0	12,935.4
1.01				
R041606_001	4	280	31,252.0	31,061.0
0.61				
R061606_001	6	430	49,421.0	48,736.0
1.39				
Average	—	—	—	—
1.32				

References

- [1] W.-J. Shin, I. Choi, S.-H. Cho, H.-J. Kim. Learning to Retrieve Containers. *Transportation Research Part C*, 2026.

- [2] W.-C. Lin et al. CRP heuristic. *Transportation Research Part E*, 2015.
- [3] K.-H. Kim et al. CRP heuristic with multiple cranes. *IJPR*, 2016.
- [4] Y. Lee, Y.-J. Lee. CRP heuristic. *OR Spectrum*, 2010.
- [5] F. Forster, A. Bortfeldt. TS for CRP. *Computers & OR*, 2012.
- [6] S. Cifuentes, M.-C. Riff. GRASP for CRP. *Expert Systems with Applications*, 2020.
- [7] M. Durasevic et al. GP for CRP. *EJOR*, 2025.
- [8] Y. Kwon et al. POMO. *NeurIPS*, 2020.
- [9] R. Stahlbock, S. Voß. OR at container terminals. *OR Spectrum*, 2008.
- [10] W. Kool et al. Attention, Learn to Solve Routing Problems! *ICLR*, 2019.
- [11] P. Legato, R. M. Mazza. Berth planning. *EJOR*, 2001.
- [12] M. Sammarra et al. Quay crane scheduling. *Journal of Scheduling*, 2007.
- [13] C. Zhang et al. Dynamic crane deployment. *Transportation Research Part B*, 2002.
- [14] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.
- [15] R. J. Williams. REINFORCE. *Machine Learning*, 1992.
- [16] M. Caserta et al. Corridor method for BRP. *OR Spectrum*, 2012.